

OpenCV Vision 과제 AI 사용 보고서

학번 : 2026406076

이름 : 정창규

제출일 : 2026-09-21

목차

1. AI 사용 목적

2. 과제 1에서 AI를 사용한 부분

- 2-1. OpenCV 및 카메라 동작 확인
- 2-2. OpenCV 이미지의 ROS2 전달 구조 확인
- 2-3. Callback과 mutex

3. 과제 2에서 AI를 사용한 부분

- 3-1. PPT의 OpenCV 코드 이해 및 적용
- 3-2. HSV 범위 조정
- 3-3. 형태학 연산
- 3-4. 결과 이미지 저장과 화면 출력

4. 느낀 점

1. AI 사용 목적

이번 OpenCV Vision 과제를 진행하면서 AI를 과제의 완성 코드를 그대로 얻기 위한 목적으로 사용하기보다는, 수업 자료에서 이해가 부족했던 개념을 확인하고 실제 구현 과정에서 발생한 오류의 원인을 찾는 용도로 사용하였다.

과제 1에서는 ROS2 환경에서 카메라 노드와 UI 노드를 하나의 패키지 안에 구현하고, 카메라에서 받은 영상을 UI에 출력해야 했다. 또한 이미지 Callback 처리와 공유 이미지에 대한 mutex 보호가 필요했다. 과제 2에서는 한 장의 이미지에서 빨강, 파랑, 초록 객체를 동시에 인식하고 각 객체에 서로 다른 색상의 Bounding Box를 표시해야 했다.

OpenCV를 처음 사용하면서 설치 확인과 카메라 입력 등 기본적인 사용 방법을 익히기 위해 AI를 활용하였고, 이후 과제를 진행하면서 이해가 부족한 부분이나 발생한 문제의 원인을 확인하는 데에도 활용하였다.

2. 과제 1에서 AI를 사용한 부분

2-1. OpenCV 및 카메라 동작 확인

과제를 시작하기 전에 Ubuntu 환경에서 OpenCV가 정상적으로 설치되어 있는지 확인하였다. Python의 cv2와 C++의 OpenCV 라이브러리 버전을 확인하였고, 노트북의 웹캠이 /dev/video0부터 /dev/video3까지 인식되는 것도 확인하였다.

이 과정에서 AI에게 Linux 환경에서 카메라 장치를 확인하는 방법과 OpenCV의 VideoCapture를 이용하여 카메라 영상을 출력하는 방법을 질문하였다.

실제 카메라 테스트를 진행하면서 처음에는 VideoCapture(0)을 실행한 직후 화면이 뜨지 않아 문제가 있는 것으로 생각했지만, 잠시 기다린 뒤 영상이 정상적으로 출력되는 것을 확인하였다. 이를 통해 카메라가 처음 실행될 때 장치 초기화에 시간이 걸릴 수 있다는 점도 알게 되었다.

우분투에서 OpenCV가 제대로 설치됐는지 확인하는 방법이란 웹캠이 어떤 /dev/video 번호로 잡혔는지 확인하는 방법 알려줘

OpenCV 설치 확인은 터미널에서:

```
</> Bash
pkg-config --modversion opencv4
```

버전이 나오면 정상 설치된 거야.

웹캠 번호 확인은:

```
</> Bash
ls /dev/video*
```

더 정확히 확인하려면:

```
</> Bash
v4l2-ctl --list-devices
```

예를 들어 /dev/video0 이면 OpenCV에서는 cv::VideoCapture cap(0); 처럼 사용하면 돼.

↓

2-2. OpenCV 이미지의 ROS2 전달 구조 확인

과제 1에서는 camera_node에서 OpenCV를 이용해 카메라 영상을 받아오고, 이를 ui_node로 전달하는 구조로 구현하였다.

camera_node에서는 cv::VideoCapture를 이용하여 카메라 프레임을 읽고, cv_bridge를 사용하여 cv::Mat 데이터를 ROS2의 sensor_msgs::msg::Image 형태로 변환한 뒤 topic으로 전달하였다.

ui_node에서는 해당 topic을 Subscribe하여 영상을 받아 화면에 출력하도록 구성하였다.

이 과정에서 OpenCV의 이미지를 ROS2 메시지로 전달할 때 cv_bridge가 어떤 역할을 하는지와 camera_node와 ui_node 사이에서 이미지가 전달되는 흐름을 AI에게 질문하여 확인하였다.

2-3. Callback과 mutex

과제 1을 구현하면서 이미지 Callback과 mutex의 사용 방법을 중점적으로 확인하였다.

UI 노드에서는 이미지 topic을 받을 때마다 imageCallback()이 실행된다. Callback 내부에서는 전달받은 이미지를 cv::Mat 형태로 변환한 뒤 공유 변수인 image_에 저장하였다.

하지만 Callback에서 image_ 값을 변경하는 동안 UI의 화면 갱신 함수가 동시에 같은 이미지에 접근하면 데이터 충돌이 발생할 가능성이 있다는 것을 알게 되었다. 이를 방지하기 위해 std::mutex와 std::lock_guard를 이용하여 공유 이미지를 보호하였다.

AI에게 mutex가 필요한 이유를 질문하면서 처음에는 단순히 "이미지를 잠그는 기능" 정도로 생각했지만, 실제로는 여러 실행 흐름이 같은 데이터에 동시에 접근하지 못하게 보호하는 기능이라는 것을 이해하였다.

또한 mutex를 오랫동안 잡고 있으면 다른 작업이 대기하게 되므로, 화면 출력까지 mutex 내부에서 처리하는 것이 아니라 이미지를 복사한 뒤 mutex를 해제하고 화면에 출력하는 방식이 더 적절하다는 점도 알게 되었다.

이미지 콜백에서 이미지를 저장하고 UI에서 같은 이미지를 읽으면 왜 mutex가 필요한 거야? \

이미지 콜백에서는 새로운 프레임을 받아서 image_에 저장하고, UI에서는 같은 image_를 읽어서 화면에 출력해.

이때 콜백이 이미지를 갱신하는 순간 UI가 동시에 같은 데이터를 읽으면 데이터가 완전히 복사되기 전에 접근할 수 있어서 충돌이나 이상한 출력이 생길 수 있어. 그래서 mutex를 사용해서 한쪽이 image_를 사용하는 동안 다른 쪽이 동시에 접근하지 못하게 보호하는 거야.

예를 들면:

```
</> C++  
  
std::lock_guard<std::mutex> lock(image_mutex_);  
image_ = received_image.clone();
```

이렇게 하면 콜백이 이미지를 저장하는 동안 UI가 같은 데이터에 접근하는 걸 막을 수 있어.

그리고 UI에서는 image_를 다른 cv::Mat에 복사한 뒤 mutex를 바로 풀고, 실제 화면 출력은 잠금 밖에서 처리하면 불필요하게 오래 막지 않을 수 있어.

3. 과제 2에서 AI를 사용한 부분

3-1. PPT의 OpenCV 코드 이해 및 적용

과제 2에서는 수업 자료에 나온 코드를 최대한 활용하여 구현하였다.

처리 과정은 다음과 같다.

이미지 입력 → HSV 변환 → `inRange()`를 이용한 색상 분리 → Opening → Closing → `findContours()` → `boundingRect()` → Bounding Box 표시

AI가 제시한 방법 중 수업 자료에 포함되지 않은 내용도 있었기 때문에, 수업에서 배운 내용을 중심으로 구현할 수 있도록 PPT에 나온 함수와 처리 순서를 기준으로 다시 확인하였다.

특히 빨간색은 HSV의 Hue 값이 범위의 처음과 끝에 나누어져 있기 때문에 두 개의 마스크를 만든 뒤 `cv::bitwise_or()`로 합치는 방식을 사용하였다.

이 부분은 처음에는 빨간색만 왜 두 개의 범위를 사용하는지 이해하지 못했지만, Hue 값이 원형으로 연결되어 있고 빨간색이 0 근처와 179 근처에 나누어 존재한다는 것을 이해하게 되었다.

OpenCV HSV에서 빨간색을 검출할 때 왜 Hue 범위를 0에서 10이랑 170에서 179 두 개로 나눠서 합쳐야 돼?

OpenCV의 HSV에서는 Hue 값이 0~179 범위로 표현돼. 그런데 빨간색은 이 범위 안에서 한쪽에만 모여 있는 게 아니라, 0 근처와 179 근처 양쪽 끝에 걸쳐서 나타나.

Hue 값은 사실 원형으로 이어지는 색상 정보라서, 빨간색에서 시작해서 노랑, 초록, 파랑 등을 지나 다시 빨간색으로 돌아오는 구조야. OpenCV에서는 이 원형 범위를 0~179로 표현하기 때문에 빨간색이 시작 부분인 0~10 근처에도 있고, 끝부분인 170~179 근처에도 존재하게 돼.

그래서 빨간색을 검출할 때는 보통 두 범위를 따로 마스크로 만들어.

```
</> C++  
cv::inRange(hsv, cv::Scalar(0, 80, 50),  
            cv::Scalar(10, 255, 255), mask1);  
  
cv::inRange(hsv, cv::Scalar(170, 80, 50),  
            cv::Scalar(179, 255, 255), mask2);
```

그리고 두 마스크를

```
</> C++  
cv::bitwise_or(mask1, mask2, mask);
```

처럼 합쳐서 하나의 빨간색 마스크로 사용해.

만약 0~10만 사용하면 170~179 쪽 빨간색을 놓칠 수 있고, 반대로 170~179만 사용해도 일부 빨간색이 빠질 수 있어. 그래서 빨간색은 두 구간을 따로 검출한 뒤 합쳐야 좀 더 안정적으로 잡을 수 있는 거야.

📄 📌 🔄 🗑️ ...

3-2. HSV 범위 조정

처음 실행했을 때 빨강, 파랑, 초록 객체는 모두 검출되었지만 Bounding Box가 PPT 예시처럼 객체의 외곽에 정확하게 맞지 않았다.

특히 파란색 공과 초록색 블록의 어두운 부분이 색상 범위에서 제외되어 Bounding Box가 실제 물체보다 좁게 생성되는 문제가 있었다.

AI와 결과 이미지를 비교하면서 원인을 확인한 결과 S와 V의 하한값이 높게 설정되어 있어 어두운 부분이 마스크에서 제외되고 있다는 것을 알게 되었다.

따라서 파랑과 초록의 HSV 범위를 실제 이미지에 맞게 조정하였고, 그 결과 객체 전체가 인식되면서 Bounding Box도 더 정확하게 표시되었다.

이 과정을 통해 HSV 값은 무조건 정해진 값을 사용하는 것이 아니라 이미지의 조명과 색상 상태에 따라 조절해야 한다는 것을 알게 되었다.

파란색 공이랑 초록색 물체의 Bounding Box가 물체 전체를 못 잡고 안쪽으로 잡히는데, 왜 이런 문제가 생기는 거야?

이런 경우는 보통 HSV 범위를 너무 좁게 잡아서 실제 물체의 일부만 검출되기 때문이야.

파란색 공이나 초록색 물체처럼 한 가지 색으로 보이는 물체도 실제 이미지에서는 모든 픽셀이 완전히 같은 HSV 값을 가지지는 않아. 빛을 직접 받는 부분은 밝아서 V 값이 높고, 그림자가 생긴 부분은 V 값이 낮아져. 반사광이나 조명 때문에 색이 벌어지는 부분은 S 값도 낮아질 수 있어.

예를 들어 `inRange()` 에서 S와 V의 최소값을 너무 높게 설정하면, 물체의 중심처럼 색이 선명하고 밝은 부분만 마스크에 남고 가장자리나 그림자 부분은 검출되지 않을 수 있어.

</> C++

```
cv::inRange(hsv, lower, upper, mask);
```

이때 범위 안에 들어온 픽셀만 흰색으로 남고, 범위에서 벗어난 부분은 검은색이 돼. 그러면 원래 물체 전체가 하나의 영역이어도 마스크에서는 일부만 남게 되는 거야.

그다음 `findContours()` 는 이 마스크에 남아 있는 흰색 영역을 기준으로 윤곽선을 찾고, `boundingRect()` 도 그 윤곽선만 기준으로 사각형을 만들어. 그래서 실제 물체는 더 큰데 Bounding Box가 물체 안쪽으로 작게 잡히는 현상이 생길 수 있어.

이런 경우에는 HSV 범위를 실제 이미지에 맞게 조금 넓혀주면 돼. 특히 S와 V의 하한값을 너무 높게 잡았다면 조금 낮춰서 어두운 부분이나 채도가 낮은 부분까지 포함되도록 조정하는 방식이 효과적이야.

즉, Bounding Box 자체의 문제가 아니라 그 전에 만들어진 색상 마스크가 물체 전체를 제대로 포함하지 못한 것이 원인이라고 보면 돼.

📄 ⬆️ ↺️ 🔍 ...

3-3. 형태학 연산

색상 마스크를 만든 뒤 Opening과 Closing 연산을 적용하였다.

Opening은 침식 후 팽창하는 연산이며 작은 흰색 노이즈를 제거하는 데 사용하였다.

Closing은 팽창 후 침식하는 연산이며 객체 내부에 생긴 작은 검은색 구멍이나 틈을 메우는 데 사용하였다.

처음에는 침식과 팽창의 순서가 헷갈렸지만 AI에게 질문하면서 다음과 같이 정리할 수 있었다.

Opening = 침식 → 팽창

Closing = 팽창 → 침식

이후 실제 코드에서도 PPT에 나온 순서대로 Opening을 먼저 수행하고 Closing을 수행하였다.

Opening은 침식 후 팽창이고 Closing은 팽창 후 침식이라고 하는데, 각각 어떤 차이가 있고 실제 이미지 처리에서는 언제 사용하는 거야?

Opening이란 Closing은 둘 다 **이진 이미지나 마스크를 정리할 때 사용하는 형태학 연산**인데, 처리 순서랑 목적이 달라.

Opening은 **침식(Erosion) → 팽창(Dilation)** 순서로 진행해. 먼저 침식으로 작은 흰색 영역이나 튀어나온 노이즈를 줄이고, 그다음 팽창으로 원래 물체 크기를 어느 정도 복원해. 그래서 보통 **작은 흰색 점이나 잡음을 제거할 때** 사용해.

반대로 Closing은 **팽창(Dilation) → 침식(Erosion)** 순서로 진행해. 먼저 팽창해서 물체 내부의 작은 검은색 구멍이나 끊어진 부분을 메운 뒤, 침식으로 크기를 다시 정리해. 그래서 **물체 내부의 빈 공간이나 작은 틈을 메울 때** 사용해.

예를 들어 색상 검출 결과에서 물체 주변에 작은 흰색 점들이 많이 생겼다면 Opening을 쓰는 게 좋고, 물체 내부에 검은 구멍이 생기거나 영역이 조금 끊겨 있다면 Closing을 사용하면 돼.

정리하면,

〈/> 일반 텍스트

Opening = 침식 → 팽창 → 작은 흰색 노이즈 제거

Closing = 팽창 → 침식 → 작은 검은 구멍이나 틈 메우기

이렇게 기억하면 가장 쉬워.

3-4. 결과 이미지 저장과 화면 출력

과제 2에서는 `cv::imshow()`를 이용하여 결과를 화면에 출력하고 `cv::imwrite()`를 사용하여 결과를 `result.png`로 저장하였다.

처음에는 `cv::waitKey(0)`을 사용하였는데, 결과 화면을 캡처하기 위해 Print Screen 키를 누르면 해당 키 입력 때문에 `waitKey(0)`이 종료되고 이미지 창까지 닫히는 문제가 발생하였다.

AI와 원인을 확인한 후 `waitKey(1)`을 반복 실행하면서 ESC 키가 입력되었을 때만 종료되도록 수정하였다.

4. 느낀 점

과제를 진행하면서 완성된 코드를 그대로 확인하기보다는, PPT의 코드와 현재 작성한 코드를 비교하면서 특정 함수의 역할이나 오류의 원인을 질문하는 방식으로 AI를 활용하였다.

특히 OpenCV에서는 `cvtColor()`, `inRange()`, `morphologyEx()`, `findContours()`, `boundingRect()`가 각각 독립적인 기능이지만 실제 객체 인식 과정에서는 하나의 처리 흐름으로 연결된다는 것을 이해할 수 있었다.

또한 ROS2에서는 단순히 영상을 출력하는 것뿐만 아니라 Publisher와 Subscriber를 통해 데이터를 전달하고, Callback에서 데이터를 받아 처리하며, 동시에 접근할 수 있는 데이터는 mutex를 이용하여 보호해야 한다는 점도 알게 되었다.

무엇보다 AI가 처음 제시한 방법이 항상 과제의 요구사항과 완전히 일치하는 것은 아니기 때문에, 수업 자료와 과제 조건을 먼저 확인한 뒤 필요한 부분만 참고해야 한다는 점을 느꼈다. 실제 과제 2에서도 처음에는 PPT에 없는 추가적인 객체 필터링 방식이 포함되었지만, 이후 PPT의 코드를 최대한 활용하는 방향으로 다시 수정하였다.

앞으로 AI를 사용할 때에도 완성된 코드를 그대로 받아 사용하는 것보다는 수업에서 배운 내용을 기준으로 개념을 확인하고, 오류가 발생한 이유를 분석하거나 이해가 부족한 부분을 질문하는 방식으로 활용하는 것이 더 도움이 될 것이라고 생각하였다.